

Python on Windows on Arm

Ecosystem Enablement Update

Diego Russo (Arm)
Gleb Khmyznikov (Microsoft)

2026-07-17

EuroPython 2023

Python on Windows on Arm (WoA)

Enablement

- Python 3.8 added experimental support for WoA
 - Download it via NuGet or build it from source
- From Python 3.11 WoA is **officially** supported
- [Joint effort to build a WoA ecosystem](#) which supports native development



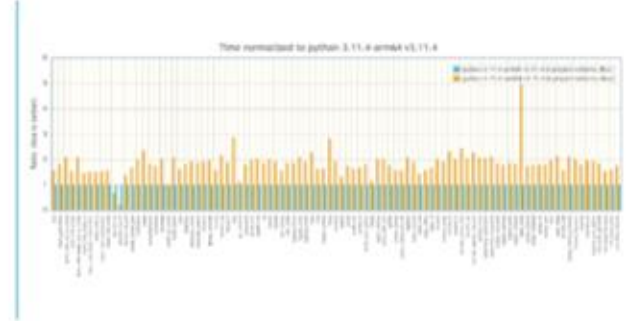
20 © 2023 Arm

arm

Python on Windows on Arm (WoA)

Performance

- Windows 11 on Arm can run x64 binaries via emulation
- For pyperformance, Python Arm64 is almost twice as fast (on average) than amd64 (emulated) version
- Recommendation:** start building native packages for this platform



21 © 2023 Arm

arm

Python on Windows on Arm (WoA)

Enablement

Tested the top [250 packages](#); over 70% succeeded

- Acted on a [subset](#): setuptools, pip, distlib, numpy, cffi, lxml, greenlet, pytorch, etc...
- But also on [third-party libraries](#): OpenBLAS, FreeType, libpng, zlib, ZeroMQ, etc...
- And [Toolchains](#): Ninja, CMake, Meson, Bazel, etc...

Linaro hosts a [buildbot worker](#) for building Python on WoA

- It's a Surface Pro X

Collaborative space for [Python maintainers on WoA](#)

- All feedback, contribution, and findings are welcome!

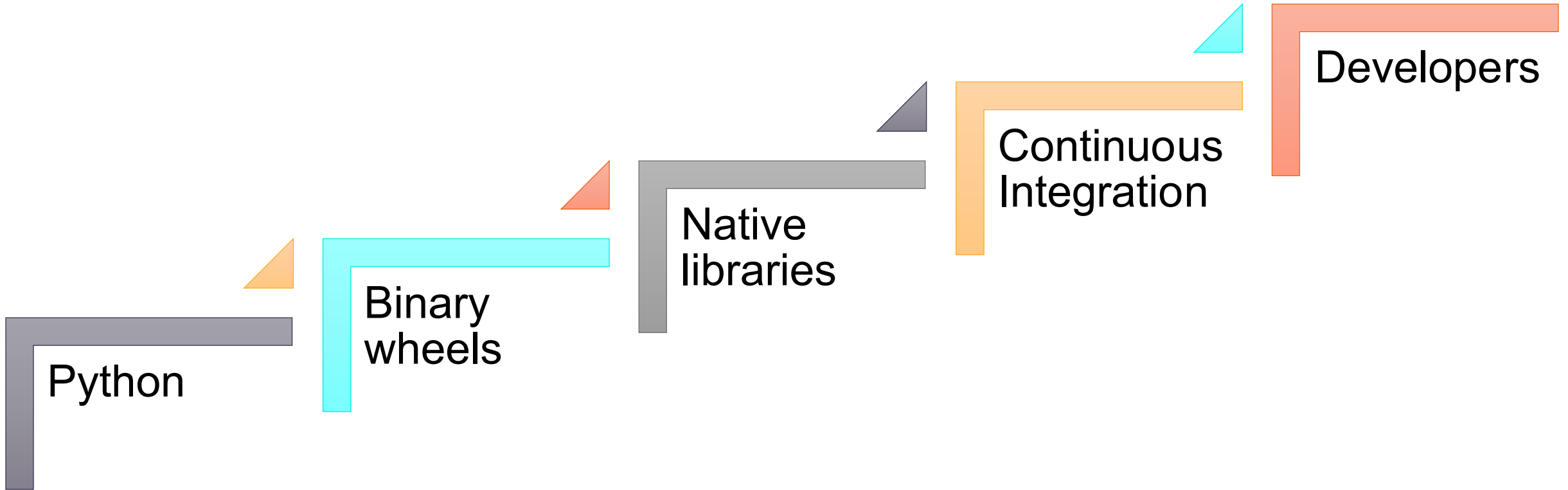
Do you want to build and test your package for WoA?

- If you have an WoA supported device: [Building Native Windows on Arm Apps with Python](#)
- If you don't have an WoA supported device
- Build: cross compilation is still the best approach
- Test: [CMake / x-Compiler - Docker to the rescue](#)

22 © 2023 Arm

arm

Supporting Python isn't enough



Continuous Integration

Native CI made Windows on Arm practical for open-source projects

Previous workflow



- ☐ x64 Emulation
- ☐ Self-hosted
- ☐ Cross Compilation
- ☐ Manual testing

Native workflow



- ☐ Native Arm64 execution
- ☐ GitHub-hosted
- ☐ Native builds
- ☐ Automated CI

Building a first-class developer platform

Built through collaboration between Arm, Microsoft and GitHub

Ready to develop

- Visual Studio 2022
- MSVC & LLVM/Clang
- CMake & Ninja
- Python, Git & PowerShell

Ready to deploy

- Native Arm64 execution
- Cross-compilation support
- Open source validated
- Production-ready CI

Continuously Improving

- Community feedback
- New tools
- Image updates
- Documentation

jobs:

build:

runs-on: windows-11-arm

Windows on Arm becomes first-class in GitHub Actions

For developers

- Native Windows Arm CI
- No infrastructure to manage
- Faster onboarding

For maintainers

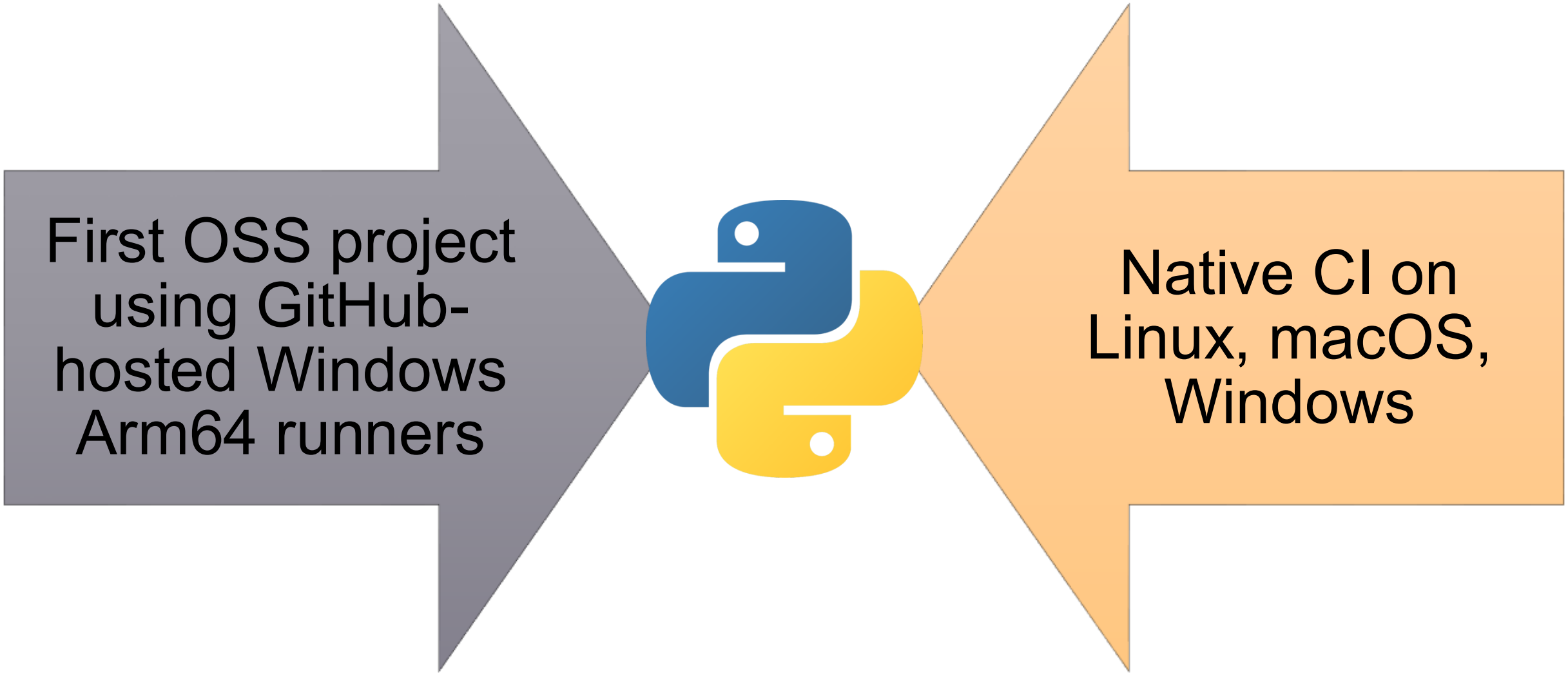
- Automated validation
- Cross-platform parity
- Reliable regression testing

For the ecosystem

- Better software quality
- More native applications
- Long-term platform support

CPython: proving the platform

One workflow. Native Arm everywhere.



The diagram illustrates a workflow for CPython. It features a central Python logo, which is a blue and yellow interlocking snake. To the left of the logo is a large grey arrow pointing right, containing the text 'First OSS project using GitHub-hosted Windows Arm64 runners'. To the right of the logo is a large orange arrow pointing left, containing the text 'Native CI on Linux, macOS, Windows'. The arrows and logo are centered horizontally, creating a visual flow from the OSS project to the native CI environment.

First OSS project
using GitHub-
hosted Windows
Arm64 runners

Native CI on
Linux, macOS,
Windows

How the ecosystem
was enabled package
by package

Visibility moved the ecosystem

87.6% → ~95%

of the top 1000 packages work natively on Windows on Arm

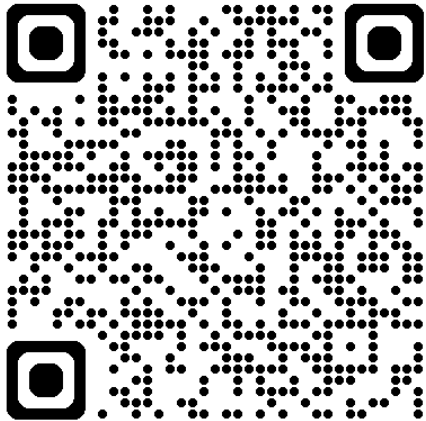
A public tracker turns status into action

- 1

Track gaps
Real compatibility, not intention
- 2

Link the work
Issues and PRs stay connected
- 3

Invite action
Maintainers and contributors can step in



Public Tracker

	pynacl	Issue	PR	-	Support dropped due to GH image issues	-	pypi
	thinc	Issue	PR	-	Only 8.x available on pypi	#28	pypi

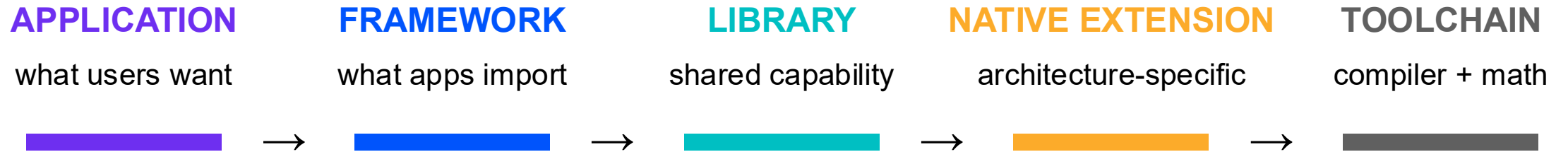
khmyznikov mentioned this pull request on Mar 16

[\[Tracker\] WoA](#) [Libs](#) [khmyznikov/PyEnv-WoA-State#1](#)

Open



One missing wheel can stall the whole stack



No native artifact → Source build → Special toolchain → Users turn back

**Fix the lowest missing layer first;
everything above it can then move together.**



The ecosystem is broad—and the remaining tail is visible

Major package categories enabled

● In progress

Scientific computing (8)

numpy · scipy · matplotlib · contourpy ·
scikit-image · pyproj · h3 · soundfile

● In progress: shapely, pywavelets,
numexpr, statsmodels, h5py

Data science (2)

pandas · duckdb

Machine learning & NLP (12)

scikit-learn · xgboost-cpu · onnx ·
spacy · thinc · tokenizers ·
sentencepiece · ml-dtypes · cymem ·
preshed · srsly · marisa-trie

● In progress: faiss-cpu, safetensors,
bitsandbytes, torchvision, blis, llvmlite

Developer tooling (8)

mypy · coverage · psutil · ddtrace ·
libcst · wrapt · logbook · cffi

Databases & connectors (3)

pymongo · pyodbc · clickhouse-
connect

● In progress: snowflake-connector-
python, sqlalchemy

Web, networking & async (10)

tornado · gevent · greenlet · winloop ·
multidict · yarl · frozenlist · propcache ·
hiredis · pycares · aiohttp

Serialization, formats & compression (11)

msgpack · orjson · ujson · jiter ·
msgspec · fastavro · avro-python3 ·
thriftpy2 · pyyaml · zstandard · zopfli
● In progress: brotli, cramjam, ijson

Text, markup & parsing (5)

lxml · markupsafe · nh3 · regex ·
google-re2
● In progress: ciso8601

Cryptography & security (4)

bcrypt · argon2-cffi-bindings ·
pycryptodome · pycryptodomex
● In progress: cryptography, pynacl

Core utilities & data structures (5)

rpds-py · bitarray · mmh3 · maxminddb ·
zope-interface

Four packages reveal the same enablement pattern



NumPy

Keystone wheels unblock everything above them.

SciPy

Specialized compiler and math dependencies must become reproducible.

pandas

Once foundations land, native builds become routine.

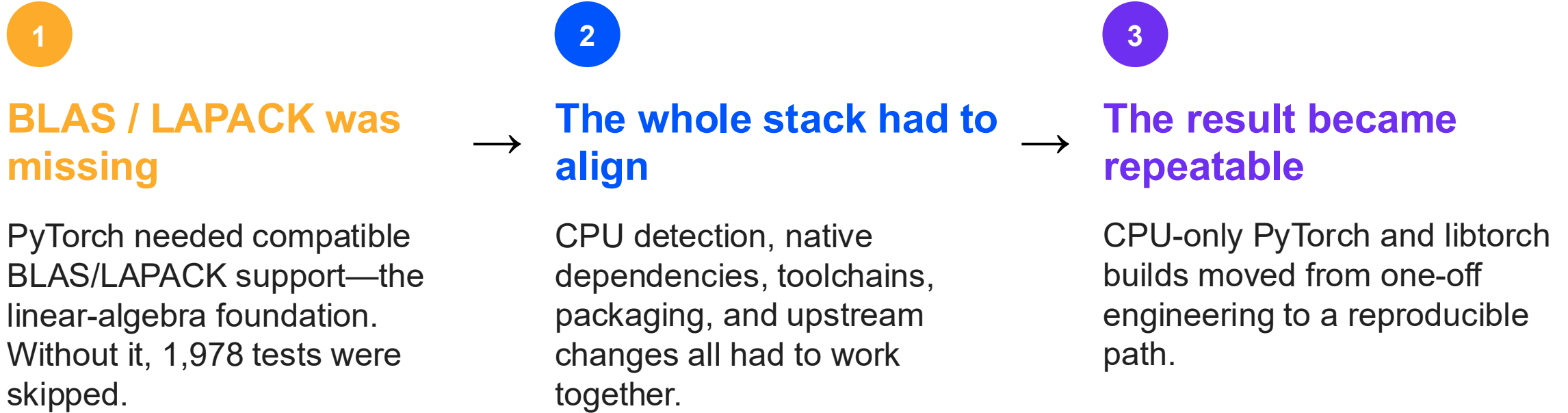
PyTorch

The full stack—dependencies, CI, tests, and packaging—must align.

Fix the base → Automate the build → Publish the artifact → Let applications follow



PyTorch exposed the hidden layers beneath Python



ECOSYSTEM IMPACT

This work strengthened shared dependencies, toolchains, packaging patterns, and native validation across the ecosystem.

Most of yesterday's platform blockers are no longer blockers for package maintainers today.



Cryptography

Regression in 20260105.41.1:
actions/setup-python with 3.14t
fails #14058

✓ Closed

[CI][Wheels] Make the win_arm64
platform silent on failures
#14346 

 Merged

[alex](#) merged 22 commits into

[pyca:main](#) from [khmyznikov:main](#)  on May 23

drop building and testing against
windows arm64 #14216

 Merged



khmyznikov commented [on May 23](#)

[@alex](#) [@reaperhulk](#) thanks for considering the trial. I'll monitor how it's going.



alex commented [on May 23](#)

Great. Thanks for working through this with us.



Add one CI target; unlock native users

FOR MAINTAINERS

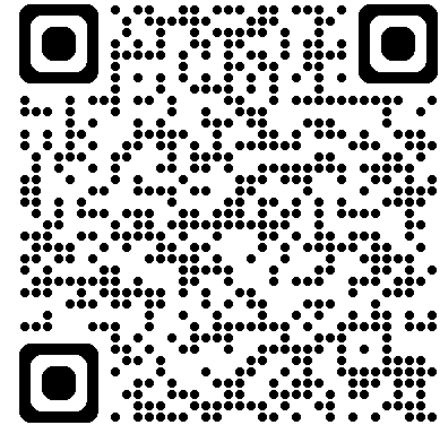
- 1 Add Windows on Arm to CI
- 2 Build and test your project
- 3 Publish win_arm64 wheels

`runs-on: windows-11-arm`

FOR DEVELOPERS

- 1 Prefer native wheels
- 2 Exercise real workloads on target hardware
- 3 Report package, version, and failure upstream

Make remaining gaps easy to reproduce.

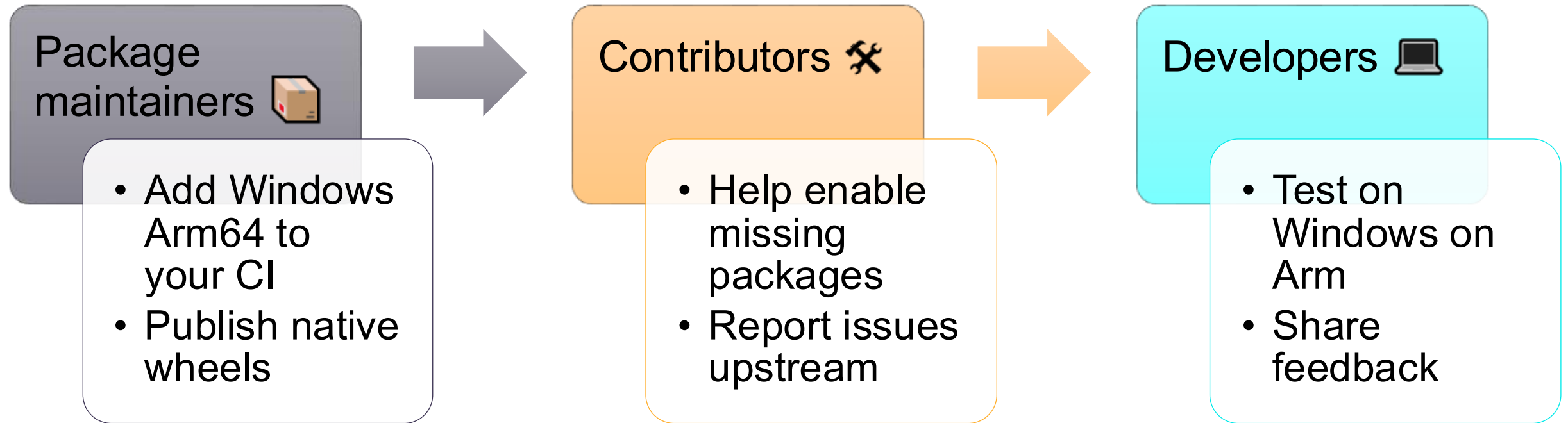


Core Collective

The infrastructure is available; community coverage is the next lever.

Windows on Arm is now a platform the Python community can confidently build on 

Every contribution moves the ecosystem forward



Continue your journey



Arm Learning Paths

Python on Windows on Arm
Step-by-step tutorials
Getting started guides



GitHub-hosted runners

Native Windows Arm64 CI
Available today



Windows App Ready

Technical guidance
Ecosystem support
Engineering collaboration



arm

Tack

ధన్యవాదగళు

Merci

Danke

Gracias

Grazie

谢谢

ありがとう

Asante

Thank you

감사합니다

ధన్యవాద

Kiitos

شكرًا

ধন্যবাদ

תודה

ధన్యవాదములు

Köszönöm